Документирование систем с микросервисной архитектурой

Роль документации в МСА; Сценарии формирования документации; Документирование разработки; Документирование развёртывания; Документирование эксплуатации; Инструменты и стандарты; Лучшие практики и антипаттерны;

Роль документации в МСА

Почему документация в МСА критически важна

В **монолитной** архитектуре вся **логика системы** сосредоточена **в единой кодовой базе**.

Разработчик, изучая исходный код, **может проследить цепочку вызовов от точки входа до хранилища данных**.

Почему документация в МСА критически важна

В микросервисной архитектуре ситуация принципиально иная:

Аспект	Монолит	МСА
Локализация логики	Единая кодовая база	Распределена по десяткам/сотням сервисов
Зависимости	Видны на уровне импорта модулей	Скрыты в сетевых вызовах, контрактах, очередях
Команда разработки	Одна или несколько команд на одном проекте	Отдельная команда (или подкоманда) на каждый сервис
Стек технологий	Как правило, единый	Гетерогенный (разные языки, фреймворки, СУБД)
Развёртывание	Единый артефакт	Множество независимых артефактов с инфраструктурным окружением

Почему документация в МСА критически важна

В МСА документация становится **не вспомогательным**, а ***первичным*** средством коммуникации между командами.

Без неё система быстро превращается в набор «чёрных ящиков», взаимодействие с которыми требует личного присутствия автора.

Цена отсутствия документации

Отсутствие или низкое качество документации в МСА приводит к следующим проблемам:

Увеличение времени онбординга нового разработчика с нескольких дней до нескольких недель;

Дублирование функциональности — команда не знает, что нужный ей сервис уже существует;

Нарушение контрактов — изменение формата сообщений или API без уведомления потребителей;

Увеличение MTTR (Mean Time To Recovery) при инцидентах — инженеры эксплуатации не понимают, как сервис должен работать в штатном режиме;

Технический долг — отсутствие зафиксированных архитектурных решений приводит к принятию противоречивых решений разными командами.

Принципы документирования в МСА

Сформулируем базовые принципы, на которых строится вся система документации:

Документация как код (Docs as Code). Документация хранится в репозитории рядом с исходным кодом, проходит код-ревью, версионируется через Git.

Близость к источнику истины. Каждый документ создаётся и поддерживается той командой, которая владеет соответствующим сервисом или компонентом.

Автоматическая генерация. Всё, что можно сгенерировать из кода, конфигурации или инфраструктуры, — генерируется автоматически.

Минимальная достаточность. Документируется то, что нельзя понять из самого кода: *почему* принято решение, *какие ограничения* существуют, *как* компоненты взаимодействуют.

Актуальность. Документация обновляется в том же pull/merge request, что и код. Устаревшая документация хуже её отсутствия.

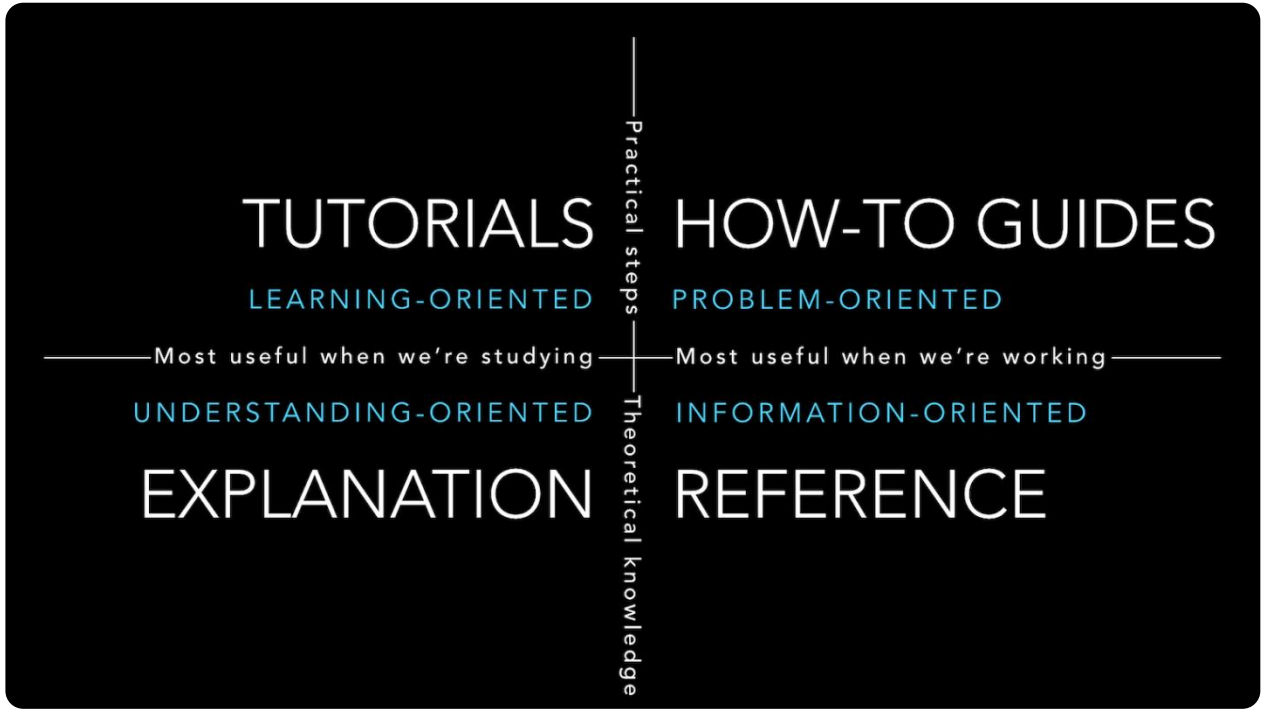
Сценарии формирования документации

Документация формируется на каждом этапе жизненного цикла системы:

Этап жизненного цикла	**Вид документации**	**Ответственный**
Проектирование	RFC, ADR, диаграммы архитектуры	Архитектор, тимлид
Разработка	README сервиса, API-спецификация, гайд контрибьютора	Команда сервиса
Тестирование	Описание тестовых сценариев, матрица совместимости	QA / разработчики
Развёртывание	Инструкции по деплою, конфигурации окружений	DevOps / SRE
Эксплуатация	Runbook, дашборды, алерт-правила, процедуры восстановления	SRE / поддержка
Вывод из эксплуатации	План декомиссии, миграции данных	Архитектор, команда сервиса

Модель «четырёх типов документации»

Современный подход к структурированию документации, предложенный Даниэле Прокки, выделяет четыре квадранта:



Модель «четырёх типов документации»

Tutorials: «Как развернуть и запустить демо-проект с тремя микросервисами за 30 минут».

How-to guides: «Как добавить новый эндпоинт в сервис заказов», «Как настроить ретраи при вызове платёжного сервиса».

Explanation: «Почему мы выбрали событийно-ориентированную модель вместо синхронных вызовов», «Как работает паттерн Saga в нашей системе».

Reference: Спецификация OpenAPI для каждого сервиса, схема сообщений в брокере, таблица конфигурационных параметров.

Сценарии инициации документации

Документация создаётся не «на всякий случай», а в ответ на конкретные события:

Создание нового сервиса → обязательный набор: README, API-контракт, схема зависимостей.

Изменение контракта (breaking change) → обновление спецификации, уведомление потребителей, changelog.

Архитектурное решение → запись ADR.

Инцидент в эксплуатации → обновление Runbook, post-mortem.

Онбординг нового члена команды → актуализация гайдов и учебных материалов.

Подготовка к аудиту / комплаенс → формализация политик безопасности, логирования, обработки данных.

Диаграммы и визуальная документация

Для описания архитектуры MCA рекомендуется использовать модель **C4** (Simon Brown), включающую четыре уровня детализации:

Уровень	Название	Что показывает	Пример для MCA
1	Context	Систему как единое целое и внешних акторов	«Интернет-магазин» + пользователи, платёжные системы
2	Container	Крупные блоки внутри системы	Сервисы, БД, брокер сообщений, API Gateway
3	Component	Внутренние компоненты одного контейнера	Модули внутри order-service
4	Code	Диаграммы классов / последовательностей	Детальная логика конкретного модуля

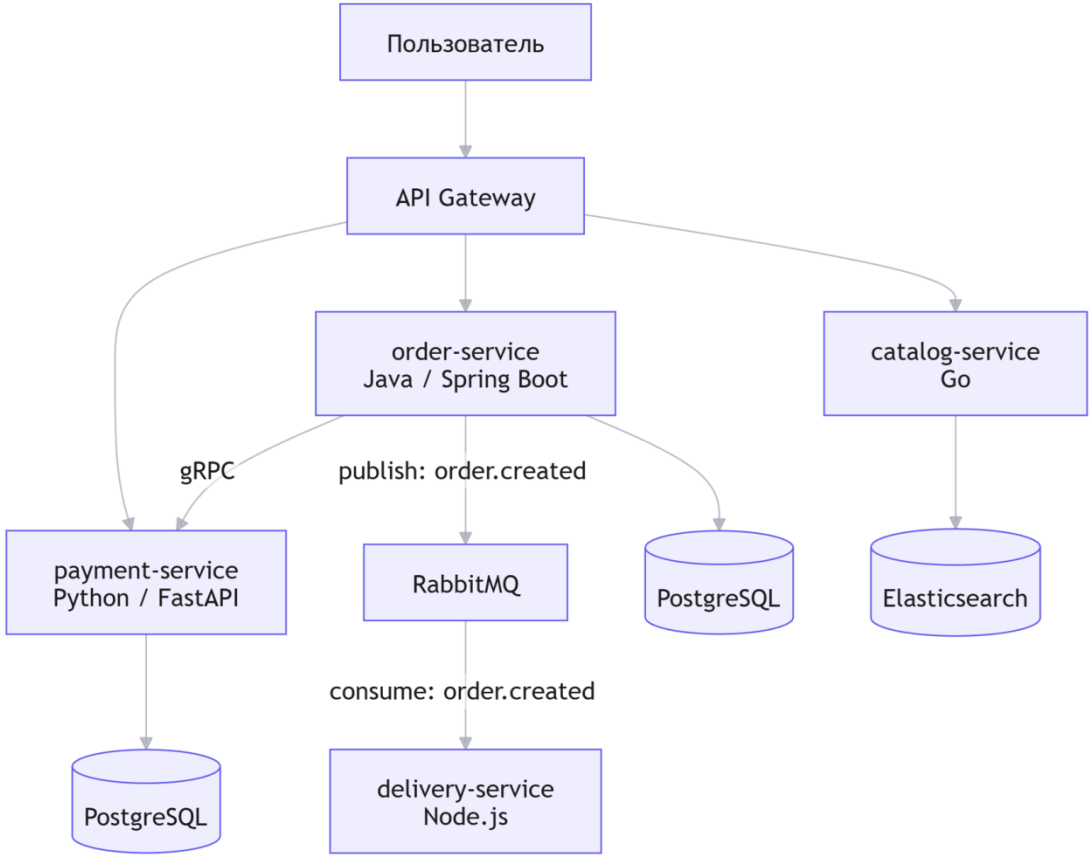
Диаграммы и визуальная документация

Для описания архитектуры МСА рекомендуется использовать модель **C4** (Simon Brown), включающую четыре уровня детализации:

Для МСА наиболее востребованы уровни **1 и 2**.

Диаграммы рекомендуется хранить в репозитории в текстовом формате:
PlantUML / **Mermaid** — текстовое описание, рендеринг в CI;
Structurizr DSL — нативная поддержка модели C4;
Draw.io / Excalidraw — для неформальных набросков (экспорт в SVG/PNG).

Пример диаграммы Container уровня (Mermaid):



Документирование бизнес-логики и доменных моделей

Документирование бизнес-логики и доменных моделей

Для каждого сервиса, реализующего сложную бизнес-логику, необходимо документировать:

Границы ограниченного контекста (Bounded Context) в терминах DDD;

Доменную модель — сущности, агрегаты, инварианты;

Бизнес-правила — валидации, переходы состояний (например, конечный автомат жизненного цикла заказа);

Глоссарий (Ubiquitous Language) — единый словарь терминов, чтобы разработчики, аналитики и менеджеры говорили на одном языке.

Инструменты и стандарты

Категория	Инструменты	Назначение
Спецификации API	OpenAPI 3.x, AsyncAPI, gRPC .proto	Контракты взаимодействия
Генерация документации	Swagger UI, Redoc, Stoplight, Scalar	Интерактивная документация из спецификаций
Docs as Code	Markdown, AsciiDoc, reStructuredText	Написание документации в репозитории
Статические генераторы	MkDocs, Docusaurus, Hugo, Sphinx	Публикация документации как сайта
Диаграммы	Mermaid, PlantUML, Structurizr, C4-PlantUML	Визуализация архитектуры
Управление документацией	Confluence, Notion, Wiki.js, GitBook	Централизованный портал документации
ADR	adr-tools, Markdown в /docs/adr/	Фиксация архитектурных решений
Changelog	Keep a Changelog, Conventional Commits	История изменений

Лучшие практики и антипаттерны

Лучшие практики

Владелец документации. Каждый документ имеет конкретного владельца (команду или человека). «Ничейная» документация быстро устаревает.

Документация в pull request. Изменение кода без обновления соответствующей документации не проходит код-ревью.

Единая точка входа. Портал документации с поиском, где любой инженер может найти нужный документ за ≤ 2 минуты.

Шаблонизация. Единые шаблоны для README, ADR, Runbook, Post-mortem снижают когнитивную нагрузку.

Версионирование. Документация версионируется вместе с релизами. Пользователь версии 1.4 видит документацию для 1.4.

Обратная связь. Кнопка «Сообщить об ошибке в документации» на каждой странице.

Регулярный аудит. Раз в квартал — ревизия документации: удаление устаревшего, обновление неточного.

Лучшие практики и антипаттерны

Антипаттерн	Описание	Последствие
«Документация потом»	Документация откладывается на «после релиза»	Никогда не создаётся
«Всё в коде»	Считается, что код самодостаточен	Невозможно понять почему принято решение
«Один большой документ»	Вся система описана в одном файле на 500 страниц	Никто не читает, невозможно поддерживать
«Копирование»	Документация дублируется между сервисами	Рассинхронизация при обновлении
«Документация ради документации»	Формальное заполнение шаблонов без реального содержания	Потеря доверия к документации
«Секретное знание»	Критичная информация хранится только в голове одного человека	Риск при уходе сотрудника (bus factor = 1)

Ключевые выводы

1. В микросервисной архитектуре документация из вспомогательного артефакта превращается в критический элемент коммуникации между десятками команд и сервисов.

2. Документация должна создаваться по принципу Docs as Code: храниться в репозитории, версионироваться, проходить ревью, генерироваться автоматически.

3. Документация охватывает весь жизненный цикл: от проектирования (ADR, RFC) через разработку (API-контракты, README) до эксплуатации (Runbook, Post-mortem).

4. Для каждого типа взаимодействия выбирается стандарт спецификации: OpenAPI для REST, AsyncAPI для событий, .proto для gRPC.

5. Документация эксплуатации (Runbook, SLO, DR-план) напрямую влияет на время восстановления при инцидентах.

6. Автоматизация генерации и проверки документации снижает нагрузку на разработчиков и повышает актуальность.

Контрольные вопросы

1. Чем принципиально отличается роль документации в MCA по сравнению с монолитной архитектурой?

2. Назовите четыре типа документации по модели Diátaxis и приведите пример каждого для MCA.

3. Какие разделы обязательно должны присутствовать в README микросервиса?

4. Чем отличается назначение спецификаций OpenAPI и AsyncAPI?

5. Что такое ADR и зачем он нужен в микросервисной системе?

6. Какие три столпа наблюдаемости (observability) документируются для эксплуатации MCA?

7. Что такое Runbook и какую структуру он имеет?

8. Почему документацию рекомендуется хранить в том же репозитории, что и код?